

SupportDesk — Full Project Closure

AI Support Intelligence Agent · GlobalFoods Inc.

Completed: May 2026 | **Phases:** 2–8 + Phase 10 Web App **Requirement:** Capstone — Design, Build, Evaluate an AI Agent

WHAT WAS BUILT

A production-grade AI support intelligence agent for GlobalFoods Inc. — a B2B supply chain company dealing with offline mobile order sync failures, \$2.4M ARR at risk, 47 duplicate orders, and 31 missing warehouse orders.

The system has two independently deployable components:

- Component 1 — Python CLI Agent** ([supportdesk_agent/](#)) Seven phases of progressive improvement. Accepts natural language queries. Calls structured tools. Maintains memory across turns. Adapts behaviour from feedback. Deployed with health checks and a four-level fallback chain.
- Component 2 — SupportDesk Web Application** ([supportdesk_agent/SupportDesk/](#)) A browser-based support portal with no build step. Seven pages. Three layers (frontend, API, intelligence). Connected to the agent via a FastAPI bridge ([api_server.py](#)) and a 118-line JavaScript patch ([agent_api.js](#)).

FINAL METRICS

Metric	Value
Total interactions logged	142
Phases completed	7 (Phases 2–8) + Phase 10
Overall grounding rate	96.5% (137/142)
Grounding rate Phase 5–8	100% (88/88)
Phase 2 grounding	83% (5/6)
Phase 3 grounding	100% (43/43)
Phase 4 grounding	84% (21/25)
Tool selection accuracy Phase 5+	100%
Q3 correct (FreshMart)	100% of Phase 5–8 runs
Q5 root cause consistent	100% of Phase 3–8 runs
Average confidence score	0.78
Failures documented	15

Metric	Value
Failures resolved	11 (73%)
Health check (Phase 8)	6/6 checks pass
Runtime errors (Phase 8 eval)	0 (errors.jsonl not created)
Fallback levels	4
Phase 2 latency	2ms
Phase 3 V2 latency	6,402ms
Phase 4 RAG latency	6,975ms
Phase 5 LangChain latency	9,849ms
Phase 6 Memory latency	11,566ms
Phase 7 Adaptive latency	13,917ms
Phase 8 eval avg latency	12,528ms

PHASE ARC — WHAT EACH PHASE PROVED

Phase 2 — Baseline rule-based agent

What was built: Keyword routing, static data loading, no LLM. **What it proved:** The data model and logging infrastructure work. Q1–Q4 answered correctly from rules. Q5 returned a hard error — no ticket ID was present. This failure became the motivating problem for Phase 3. **Grounding:** 83%. **Latency:** 2ms.

Phase 3 — LLM integration with prompt strategies

What was built: Three prompt strategies compared (V1 direct, V2 structured JSON, V3 chain-of-thought). V2 selected as default. **What it proved:** Language understanding works at scale. Q5 now answered with idempotency cited and 9 tickets. Q3 still wrong — LLM ranked QuickShip instead of FreshMart. \$9.1M ARR hallucination documented and traced to missing deduplication. V2 selected over V3 despite lower quality because 100% JSON compliance at 6,402ms beats an 8-point quality gain at 14,266ms with 20% parse failure rate. **Grounding:** 100%. **Latency:** 6,402ms (V2 default).

Phase 4 — RAG retrieval with ChromaDB

What was built: Five vector collections. Semantic retrieval replaced static context injection. **What it proved:** Focused retrieval beats full-context injection for synthesis queries. Q5 improved to 9 tickets cited vs 2 in Phase 3. Q3 still wrong — customer embeddings too similar for LLM differentiation. Grounding dropped to 84% because of JSON parse failures on 4 entries. The problem was structural: LLM-based ranking fails when embeddings cannot disambiguate customers. **Grounding:** 84%. **Latency:** 6,975ms.

Phase 5 — LangChain tool calling

What was built: Five schema-defined tools. `rank_customer_risk` made deterministic — Python computes and pre-sorts before the LLM sees anything. Hard-stop added on missing ticket ID. **What it proved:**

Deterministic code plus LLM narrative is more reliable than LLM ranking from embeddings. Q3 fixed permanently — FreshMart returned on 100% of runs with risk_score 78.5 CRITICAL and `deterministic:true`. TKT-9999 hallucination fixed. Tool selection accuracy: 100%. **Key architectural decision:** The LLM handles language. Python handles computation. This boundary was arrived at by failure, not by design. **Grounding:** 100%. **Tool accuracy:** 100%.

Phase 6 — Memory and multi-turn conversation

What was built: `ConversationBufferWindowMemory` (k=6) injected into every prompt. Long-term memory persisted to `long_term_memory.json`. **What it proved:** Conversation continuity changes the agent from a question-answering system into a conversational reasoning system. `history_len` grew from 2 to 10 across a five-query session. "Tell me more about their tickets" resolved FreshMart from Turn 1 context. `reset memory` cleared short-term, preserved long-term — confirmed by `history_len:2` and `tool_called:none` on post-reset pronoun query. **Grounding:** 100%. **Latency:** 11,566ms.

Phase 7 — Adaptive behaviour from feedback

What was built: Feedback rating ingested at session start. `depth_preference` injected into system prompt. Tool count and response depth adapt from a single 2/5 rating. **What it proved:** Feedback → system prompt injection → behaviour change is synchronous, persistent, and stable. A rating of 2/5 on the first query changed `tools:1→2` on the second query in the same session. `depth:detailed` persisted across three independent eval runs. The adaptation was loaded from `long_term_memory.json` before the first query — not accumulated during the session. **Grounding:** 100%. **Latency:** 13,917ms.

Phase 8 — Deployment readiness

What was built: Health check validating 6 dependencies. Walk-up `.env` loader searching 8 directory levels. Latency capture to `latency_metrics.jsonl`. Four-level fallback chain: adaptive → langchain → llm → baseline. **What it proved:** The system can be validated, monitored, and deployed with a guaranteed answer at every fallback level. All 6 health checks pass. `errors.jsonl` was not created — zero runtime errors in the eval run. **Health check:** 6/6 pass. **Runtime errors:** 0.

Phase 10 — Web application + agent bridge

What was built: SupportDesk SPA (7 pages, 3 layers, no build step). `api_server.py` FastAPI bridge (5 endpoints mapping web app calls to agent tools). `agent_api.js` (118 lines patching AI_API if server is running). **What it proved:** A CLI agent built across 7 phases can become a web intelligence layer without changing a single line of agent code. The bridge is one file. The patch is one script tag. The seam is invisible in both modes.

THE FIVE TOOLS — FINAL STATE

Tool	Input	Key behaviour	Phase fixed
<code>analyze_ticket</code>	<code>{ticket_id}</code>	Hard stop on missing ID · <code>not_found:true</code> + suggestions	Ph5

Tool	Input	Key behaviour	Phase fixed
<code>detect_patterns</code>	<code>{window, min_frequency}</code>	ChromaDB retrieval · tag clustering · root cause signal	Ph5
<code>rank_customer_risk</code>	<code>{sort_by, top_n}</code>	Deterministic Python sort · FreshMart always first · <code>deterministic:true</code>	Ph5
<code>predict_sla_risk</code>	<code>{lookahead_hours}</code>	Rule engine computes <code>hours_until_breach</code> from actual <code>due_date</code>	Ph5
<code>cross_ticket_analysis</code>	<code>{query}</code>	RAG 12 docs · <code>shared_root_cause</code> · ARR deduped · <code>retrieval_grounded:true</code>	Ph5

All tools wrapped by `tool_safeguards.py: ToolCallTracker` (`MAX_CALLS_PER_TOOL=2`, `MAX_CALLS_TOTAL=5`), `check_tool_input()`, `validate_tool_output()`.

FAILURES DOCUMENTED AND RESOLVED

Failure	Phase detected	Phase resolved	Resolution
Q5 hard error — no ticket ID	Phase 2	Phase 3	LLM answered Q5 from context
Q3 wrong customer — LLM ranked QuickShip	Phase 3	Phase 5	Deterministic Python sort
\$9.1M ARR hallucination	Phase 3	Phase 5	ARR deduped per customer
TKT-9999 hallucination — <code>docs[:1]</code> fallback	Phase 5	Phase 5	Hard stop on missing ticket ID
Phase 4 grounding 84% — JSON parse fails	Phase 4	Phase 5	Schema validation + V2 prompt
<code>window</code> param shadowing global in <code>ai.js</code>	Phase 10	Phase 10	Renamed to <code>timeWindow</code>
Insights Hub blank — parameter collision	Phase 10	Phase 10	<code>renderInsightsShell()</code> fix

Unresolved (documented):

- Latency above 8s target from Phase 5 onwards — LangChain + memory overhead
- `ConversationBufferWindowMemory` deprecation warning — migration to `RunnableWithMessageHistory` required before production
- Phase 4 grounding retrospectively 84% — not fixed at Phase 4, superseded by Phase 5
- Feedback applied globally not per-tool — per-tool mapping not implemented

THREE DECISIONS THAT DEFINED THE SYSTEM

1. Deterministic ranking over LLM ranking

Every LLM-based customer ranking across Phases 3 and 4 returned QuickShip. The root cause was structural — all customers have similar ticket types, so their embeddings are too similar for the LLM to differentiate by risk. Moving the ranking computation to Python (Phase 5) fixed Q3 permanently. The LLM's role was reduced to adding narrative on a pre-sorted result. This is the right architectural boundary: use the LLM for language, use deterministic code for computation. The boundary was arrived at by failure.

2. V2 structured JSON as default over V3 chain-of-thought

V3 scored 8 percentage points higher on quality but had 2.4× higher latency and a documented duplication bug on Q5. V2 had 100% JSON compliance across 23 consecutive runs. In a production support tool, a reliable answer in 6 seconds beats a slightly better answer in 14 seconds with a 20% parse failure rate. Reliability was weighted over quality at the same tier.

3. One script tag for the web integration

`agent_api.js` loads last, after `ai.js` defines `AI_API`. It makes one health check call. If the server responds, it patches the 5 functions in place. If not, it exits silently. `ai.js` runs as normal. The user sees nothing different. No configuration. No error handling in the UI. No feature flag. The agent and the web app remain independently deployable and independently testable.

THE LOG LINE THAT CAPTURES THE WHOLE PROJECT

```
query:          "Tell me more about their open tickets"    ← no customer named
tool_called:    detect_patterns, cross_ticket_analysis      ← two tools, multi-step
history_len:    4                                           ← FreshMart from Turn 1
depth:         detailed                                     ← adapted from rating 2/5
latency_ms:     28,203                                       ← cost of doing it right
```

In Phase 2, this query triggered a keyword routing miss and returned nothing useful. In Phase 8, it returned a complete pattern analysis — root cause, 6 ticket IDs, \$2.4M ARR impact, specific recommended action — from a single pronoun and a prior conversation turn.

That progression is what the capstone built.

WHAT THE CAPSTONE REQUIREMENT ASKED FOR — WHAT WAS DELIVERED

Requirement	Delivered
Working AI agent	Python CLI agent, Phases 2–8
Problem framing document	GlobalFoods scenario, 5 evaluation queries, success criteria

Requirement	Delivered
Demo script (3–5 forced interactions)	5 evaluation queries with full grounding evidence
Evaluation report	<code>evaluation/phase9_results.md</code> , <code>phase9_eval_report.json</code> , 19 test scenarios
Engineering justification	Phase write-ups, decision documentation, failure analysis
LangChain framework	Phase 5–8 tool orchestration, Phase 7 memory integration
Safety requirements	4 safety blocks tested, <code>tool_safeguards.py</code> , <code>safety.py</code>
Deployment readiness	<code>health_check.py</code> , fallback chain, <code>latency_metrics.jsonl</code>

Beyond requirements:

- RAG with ChromaDB (5 collections, semantic retrieval)
- Adaptive behaviour from feedback (Phase 7)
- Full-stack web application (Phase 10)
- FastAPI bridge connecting web app to agent
- 142 logged interactions with grounding metadata
- Two video recordings (Intelligence Layer + Full UI Demo)

FINAL STATEMENT

A capstone that started as a DevRev SE interview prep exercise grew into a seven-phase Python agent, a full-stack web application, a FastAPI bridge, and a deployment-ready system with health checks, fallback chains, and persistent memory. Every phase fixed the most significant remaining failure of the prior one. Every failure was documented before it was fixed.

The web app existed before the agent. The agent existed before the bridge. The bridge connected them without changing either.

The architecture in one sentence: the LLM handles language, deterministic Python handles computation, ChromaDB handles retrieval, LangChain handles orchestration, the fallback chain handles failure, the web app handles the user — and every one of those boundaries was arrived at by failure, not by design.